

Model Evaluation, Data Leakage, and Reproducibility

AI Bootcamp in Biotechnology – Rutgers and KAIMRC Academy

W. Evan Johnson, Ph.D.
Professor, Division of Infectious Disease
Director, Center for Data Science
Associate Director, Center for Biomedical Informatics and Health AI
Rutgers University – New Jersey Medical School

2026-09-15

Evaluating a model

Training and test sets

A machine learning algorithm is evaluated on how it performs in the real world with completely new datasets. We typically split the data into two parts and act as if we don't know the outcome for one of these. We refer to the group for which we know the outcome, and use to develop the algorithm, as the **training** set. We refer to the group for which we pretend we don't know the outcome as the **test** set.

The confusion matrix

Generally speaking, overall accuracy can be a deceptive measure. We construct the *confusion matrix*. We can do this in R using the function `table`:

```
table(predicted = y_hat, actual = test_set$sex)
```

```
##           actual
## predicted Female Male
##   Female     48   32
##   Male      71  374
```

The confusion matrix

If we study this table closely, it reveals a problem. If we compute the accuracy separately for each sex, we get:

```
test_set %>%  
  mutate(y_hat = y_hat) %>%  
  group_by(sex) %>%  
  summarize(accuracy = mean(y_hat == sex))
```

```
## # A tibble: 2 x 2  
##   sex      accuracy  
##   <fct>      <dbl>  
## 1 Female    0.403  
## 2 Male     0.921
```

The confusion matrix

So when computing overall accuracy, the high percentage of mistakes made for females is outweighed by the gains in correct calls for men. **This can actually be a big problem in machine learning.**

Sensitivity and specificity

We name the four entries of the **confusion matrix**:

	Actually Positive	Actually Negative
Predicted positive	True positives (TP)	False positives (FP)
Predicted negative	False negatives (FN)	True negatives (TN)

Sensitivity and specificity

Sensitivity is typically quantified by $TP / (TP + FN)$, the proportion of actual positives (the first column = $TP + FN$) that are called positives (TP). This quantity is referred to as the **true positive rate** (TPR) or **recall**.

Sensitivity and specificity

Specificity is defined as $TN/(TN + FP)$ or the proportion of negatives (the second column = $FP + TN$) that are called negatives (TN). This quantity is also called the true negative rate (TNR).

Sensitivity and specificity

The caret function `confusionMatrix` computes all these metrics for us once we define what category “positive” is. The function expects factors as input, and the first level is considered the positive outcome or $Y = 1$. In our example, Female is the first level because it comes before Male alphabetically. If you type this into R you will see several metrics including accuracy, sensitivity, specificity, and PPV.

```
cm <- confusionMatrix(data = y_hat,  
                       reference = test_set$sex)
```

Cross-validation

Over-training

Over-training or **over-fitting** results in having higher accuracy in the train set compared to the test set:

```
y_hat_knn <- predict(knn_fit, mnist_27$train, type = "class")  
confusionMatrix(y_hat_knn, mnist_27$train$y)$overall["Accuracy"]
```

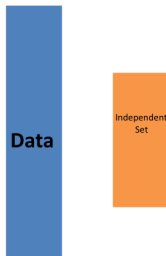
```
## Accuracy  
## 0.85875
```

```
y_hat_knn <- predict(knn_fit, mnist_27$test, type = "class")  
confusionMatrix(y_hat_knn, mnist_27$test$y)$overall["Accuracy"]
```

```
## Accuracy  
## 0.815
```

K-fold cross validation

The first one we describe is **K-fold cross validation**. A machine learning challenge starts with a dataset (blue). We need to use this to build an algorithm that will be used in an independent validation dataset (yellow).



K-fold cross validation

We create a **training set** (blue) and a **test set** (red). We will train our algorithm exclusively on the training set and use the test set only for evaluation purposes.



The caret train function

The **caret** package also provides a function that performs cross validation for us. Here we provide some examples showing how we use this incredibly helpful package. We will use the 2 or 7 example to illustrate:

```
library(tidyverse)
library(dslabs)
data("mnist_27")
```

Bias and variance

Bias-Variance tradeoff

In statistics and machine learning, the **bias–variance tradeoff** is the property of a model that the variance of the parameter estimated across samples can be reduced by increasing the bias in the estimated parameters.

The **bias–variance dilemma** or **bias–variance problem** is the conflict in trying to simultaneously minimize these two sources of error that prevent supervised learning algorithms from generalizing beyond their training set

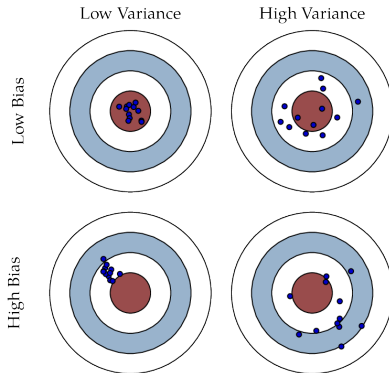
(Source: Wikipedia)

Bias-Variance tradeoff

The **bias** is an error from faulty assumptions or mispecification of the learning algorithm. High bias can cause an algorithm to miss the relevant relations between features and target outputs (underfitting).

The **variance** is an error from sensitivity to small fluctuations in the training set. High variance may result from an algorithm modeling the random noise in the training data (overfitting).

Bias-Variance tradeoff



Data leakage

What is data leakage?

Information that would **not be available at prediction time** leaks into model fitting. The performance estimate then measures something other than generalization. *Anything computed using samples that are supposed to be held out has leaked.*

Where it hides

- ▶ **Preprocessing on everything** – normalization, imputation, PCA, or **batch correction** fit on train and test together
- ▶ **Feature selection outside the loop** – ranking genes on all samples, then cross-validating only the classifier
- ▶ **The outcome itself** – a predictor that is a *consequence* of the outcome (drugs prescribed after diagnosis)
- ▶ **Groups** – the same patient in both training and test
- ▶ **Site or batch** – cases from one site, controls from another

Two questions, and a tell

1. Would I have this variable, in this form, at the moment of prediction?
2. Did any step see a sample it was not supposed to see?

And the tell: on a hard clinical problem,
a held-out AUC of 0.99 is a bug report.

What leakage buys you

Randomize the TB outcome so there is **nothing to find**, add noise genes so $p \gg n$, then select the top 10 genes and cross-validate:

	AUC
Genes chosen on all samples (leaky)	≈ 0.80
Genes chosen inside each fold (honest)	≈ 0.50

A signature manufactured entirely out of noise. The code is in `workshop_3` – run it yourself.

The fix

Every data-dependent step goes **inside** the resampling loop.

- ▶ `preProcess` within `caret::train()`
- ▶ recipes in `tidymodels`
- ▶ Pipeline in `scikit-learn`
- ▶ Nested cross-validation when you are also tuning

If a step touches the data, it belongs in the loop – the rule is the same in either language.

Reproducibility

Some terminology

- ▶ **Reproducible:** a result can be recreated by others using the same data and the same analysis pipeline
- ▶ **Replicable:** the same scientific conclusion is reached using independent data and (maybe) an independent pipeline

⇒ The reproducibility crisis is a *replication* crisis

Reproducibility Crisis

Letter | Published: 27 August 2018

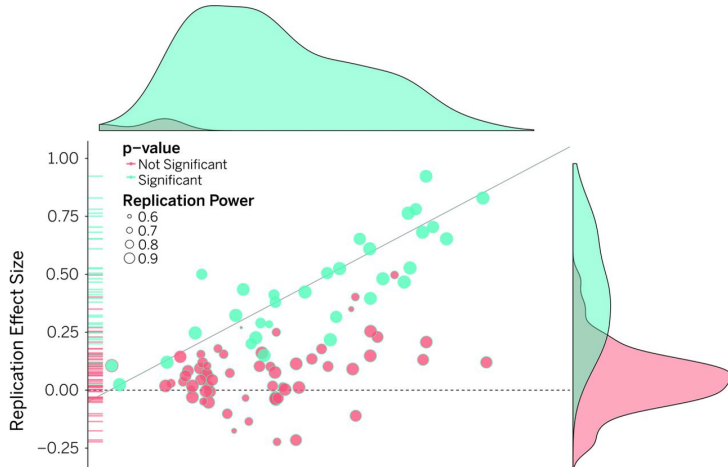
Evaluating the replicability of social science experiments in *Nature* and *Science* between 2010 and 2015

Colin F. Camerer, Anna Dreber, Felix Holzmeister, Teck-Hua Ho, Jürgen Huber, Magnus Johannesson, Michael Kirchler, Gideon Nave, Brian A. Nosek , Thomas Pfeiffer, Adam Altmejd, Nick Buttrick, Taizan Chan, Yiling Chen, Eskil Forsell, Anup Gampa, Emma Heikensten, Lily Hummer, Taisuke Imai, Siri Isaksson, Dylan Manfredi, Julia Rose, Eric-Jan Wagenmakers & Hang Wu

Nature Human Behaviour **2**, 637–644 (2018) | [Download Citation](#) 

Reproducibility Crisis

Original study effect size versus replication effect size (correlation coefficients)



Non-Reproducible Science

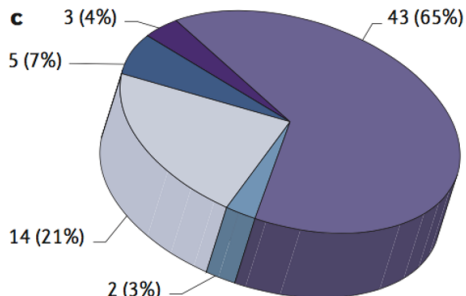
Correspondence

Nature Reviews Drug Discovery **10**, 712 (September 2011) | doi:10.1038/nrd3439-c1

Believe it or not: how much can we rely on published data on potential drug targets?

See also: [News and Analysis by Arrowsmith](#)

Florian Prinz¹, Thomas Schlange² & Khusru Asadullah³



A cautionary tale: genomic signatures for cancer

Using the NCI60 to Predict Sensitivity

Genomic signatures to guide the use of
chemotherapeutics

Anil Potti^{1,2}, Holly K Dressman^{1,3}, Andrea Bild^{1,3}, Richard F Riedel^{1,2}, Gina Chan⁴, Robyn Sayer⁴,
Janiel Cragun⁴, Hope Cottrill⁴, Michael J Kelley², Rebecca Petersen⁵, David Harpole⁵, Jeffrey Marks⁵,
Andrew Berchuck^{1,6}, Geoffrey S Ginsburg^{1,2}, Phillip Febbo¹⁻³, Johnathan Lancaster⁴ &
Joseph R Nevins¹⁻³

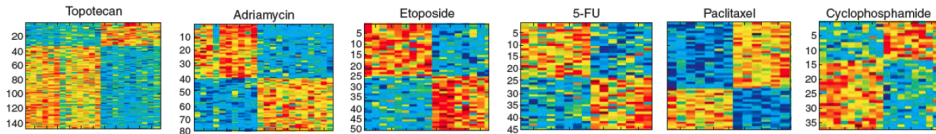
Potti et al (2006), Nature Medicine, 12:1294-1300.

(Credit: Keith Baggerly and Kevin Coombes)

A cautionary tale: genomic signatures for cancer

The claim: microarray data from cell lines (the NCI60) can define drug response “signatures” that predict whether patients will respond.

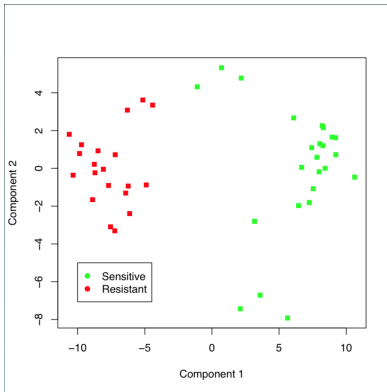
Examples were given for 7 commonly used agents.



This got the research community very excited – and moved into clinical trials.

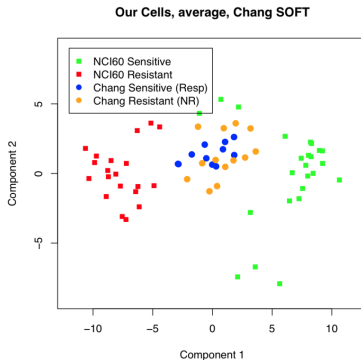
A cautionary tale: genomic signatures for cancer

We want the test data to look like this:



A cautionary tale: genomic signatures for cancer

But instead it looked like this:



A cautionary tale: genomic signatures for cancer

The Lung Metagene Score (LMS)

The NEW ENGLAND JOURNAL of MEDICINE

ORIGINAL ARTICLE

A Genomic Strategy to Refine Prognosis in Early-Stage Non-Small-Cell Lung Cancer

Anil Potti, M.D., Sayan Mukherjee, Ph.D., Rebecca Petersen, M.D.,
Holly K. Dressman, Ph.D., Andrea Birk, Ph.D., Jason Koontz, M.D.,
Robert Kratzke, M.D., Mark A. Watson, M.D., Ph.D., Michael Kelley, M.D.,
Geoffrey S. Ginsburg, M.D., Ph.D., Mike West, Ph.D., David H. Harpole, Jr., M.D.,
and Joseph R. Nevins, Ph.D.

One of “the most significant advances on the front lines of cancer.” – Ozols et al., JCO, 25:146-62, 2007, ASCO survey of 2006.

A cautionary tale: genomic signatures for cancer

 Open Access

 Select Language ▼

[Translator Disclaimer](#)

December 2009

Deriving chemosensitivity from cell lines: Forensic bioinformatics and reproducible research in high-throughput biology

[Keith A. Baggerly](#), [Kevin R. Coombes](#)

Ann. Appl. Stat. 3(4): 1309-1334 (December 2009). DOI: 10.1214/09-AOAS291

ABOUT

FIRST PAGE

CITED BY

REFERENCES

SUPPLEMENTAL
CONTENT

Abstract

High-throughput biological assays such as microarrays let us ask very detailed questions about how diseases operate, and promise to let us personalize therapy. Data processing, however, is often not described well enough to allow for exact reproduction of the results, leading to exercises in “forensic bioinformatics” where aspects of raw data and reported results are used to infer what methods must

Real-life consequences

Real-life consequences

Hydroxychloroquine or chloroquine with or without a macrolide for treatment of COVID-19: a multinational registry analysis

Mandeep R Mehra, Sapan S Desai, Frank Ruschitzka, Amit N Patel

The Lancet

New
England
Journal of
Medicine

ORIGINAL ARTICLE

Cardiovascular Disease, Drug Therapy,
and Mortality in Covid-19

Leakage is a leading cause

Kapoor & Narayanan (*Patterns*, 2023) surveyed 294 papers across 17 fields and found data leakage to be a leading driver of machine learning results that fail to replicate.

The two problems are one problem.

Why bother?

Markowitz *Genome Biology* (2015) 16:274
DOI 10.1186/s13059-015-0850-7



COMMENT

Open Access

Five selfish reasons to work reproducibly



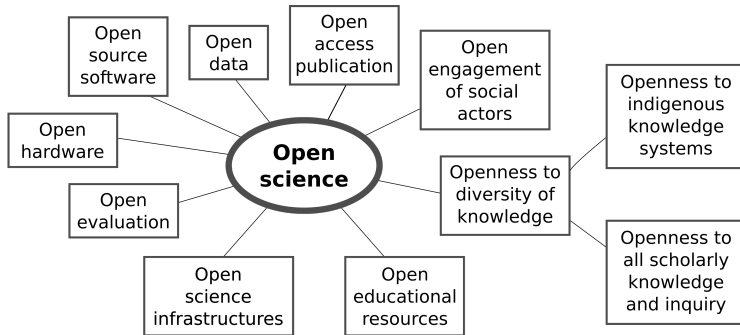
Florian Markowitz

1. Reproducibility helps to avoid disaster
2. Reproducibility makes it easier to write papers
3. Reproducibility helps reviewers see it your way
4. Reproducibility enables continuity of your work
5. Reproducibility helps to build your reputation

Some cool tools for reproducible science

- ▶ **RMarkdown / Jupyter Notebooks**: dynamic documents that combine code, visualizations, narrative text, and output in one reproducible report.
- ▶ **GitHub**: version control and collaboration for code, documents, and workflows.
- ▶ **Nextflow / Snakemake**: workflow management for scalable, portable, multi-step pipelines.
- ▶ **Docker / Singularity**: containerization for consistent computational environments, so your code runs anywhere.
- ▶ **R/Shiny**: interactive applications and dashboards for data exploration.

Open Science



(Open science elements based on UNESCO presentation of 17 February 2021)

Open Science Practices (Johnson Lab)

Data, Code and Software

Data: Shared in website links, public repositories (preferred)

Code: Websites, supplementary files, GitHub (preferred)

Software: Make an R package! (or Python, etc.) Share on GitHub or Bioconductor

Hands-on practice

Everything in this session has a matching workshop you can work through at your own pace:

`workshop_3`

- ▶ Data partitioning and `trainControl`
- ▶ ROC curves and AUC across six algorithms
- ▶ Regularization with `glmnet`: LASSO, ridge, elastic net

All workshops are in the course repository, with the R Markdown source and a rendered HTML version.

Session Info

```
sessionInfo()
```

```
## R version 4.5.2 (2025-10-31)
## Platform: aarch64-apple-darwin20
## Running under: macOS Tahoe 26.6.2
##
## Matrix products: default
## BLAS: /System/Library/Frameworks/Accelerate.framework/Versions/A/Frameworks/vecLib.framework/Versions/A/libBLAS.dylib
## LAPACK: /Library/Frameworks/R.framework/Versions/4.5-arm64/Resources/lib/libRlapack.dylib; LAPACK version 3.12.1
##
## locale:
## [1] en_US.UTF-8/en_US.UTF-8/en_US.UTF-8/C/en_US.UTF-8/en_US.UTF-8
##
## time zone: Asia/Qatar
## tzcode source: internal
##
## attached base packages:
## [1] stats      graphics  grDevices utils      datasets  methods   base
##
## other attached packages:
## [1] pROC_1.19.0.1  gridExtra_2.3  glmnet_5.0      Matrix_1.7-5
## [5] dslabs_0.9.1   caret_7.0-1    lattice_0.22-9  lubridate_1.9.5
## [9] forcats_1.0.1  stringr_1.6.0  dplyr_1.2.1     purrr_1.2.2
## [13] readr_2.2.0    tidyr_1.3.2    tibble_3.3.1    ggplot2_4.0.3
## [17] tidyverse_2.0.0
##
## loaded via a renv session (and not attached):
```